BIT 4107 – Mobile Application Development

ENHANCED PROFESSIONAL STUDENT LOGBOOK

Student Name: Joseph Ahadi

Admission Number: **BIT/2024/54480**

School/Faculty: Computing and informatics

Programme: _Mobile application development

Academic Year: 2026

Semester: ii

Lecturer: _Michael Nyoro

Phone Number: 0794505157

Week 1: Introduction to Mobile Application Development & Week 2: Mobile Development Environment

Practical work done

Android studio environment set up

Setting up first android studio project

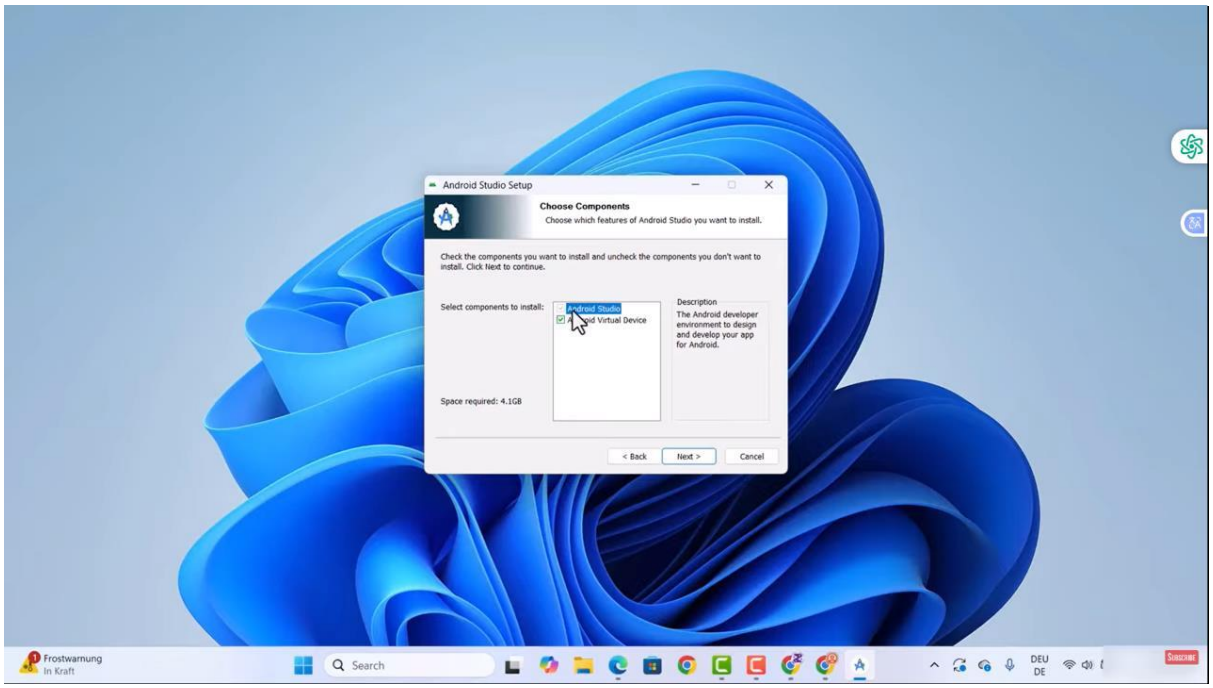
Learned how frameworks work and how UI/UX can be used

Learned how AI Agent works in referencing the code in the code window

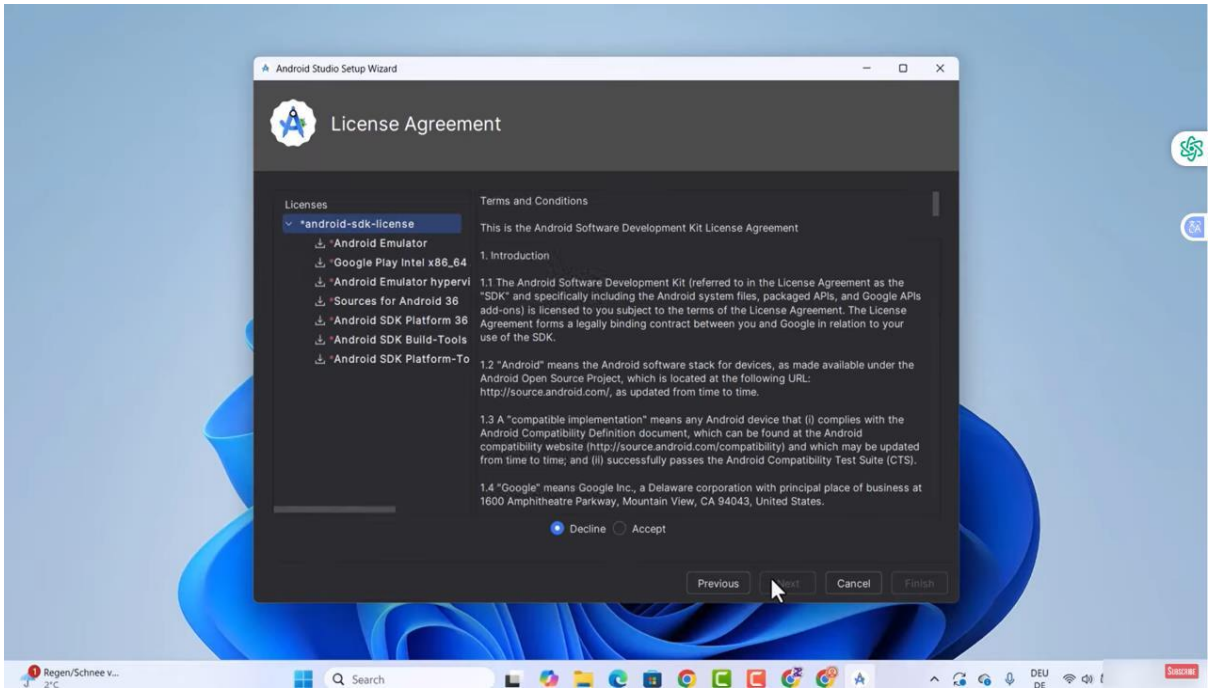
Learned how to use some basic flutter language

The screenshots below will show evidence of the processes that i followed up to the latest updates in GitHub

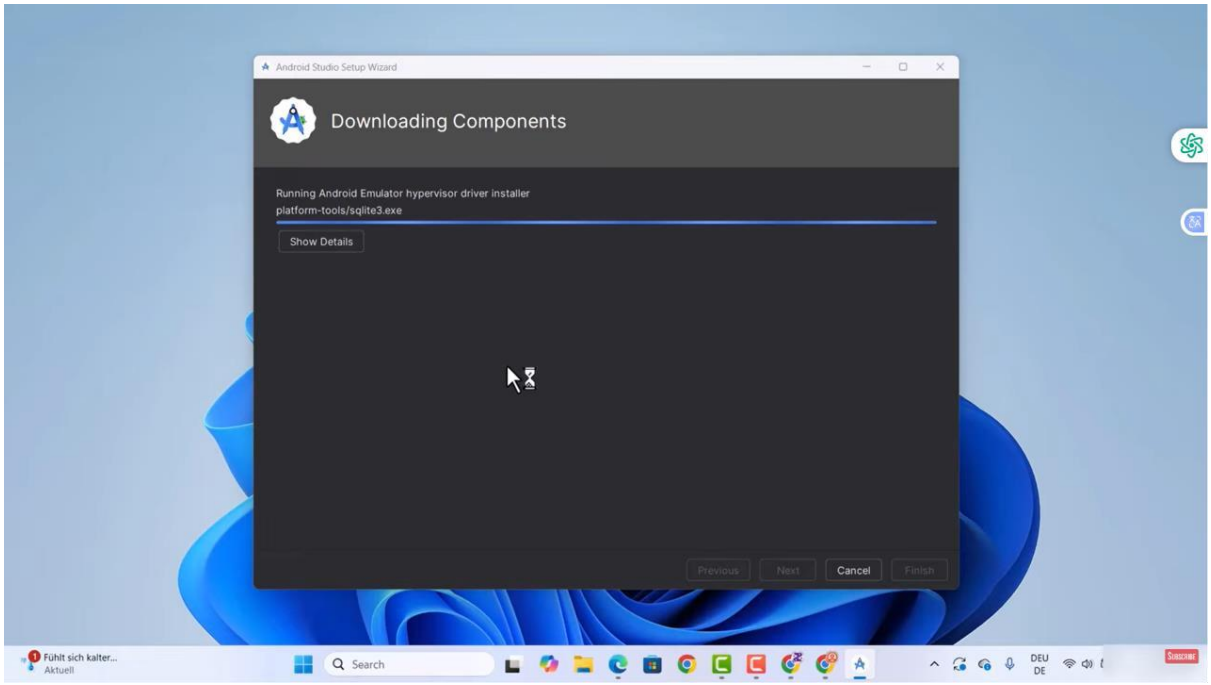
Android studio installation



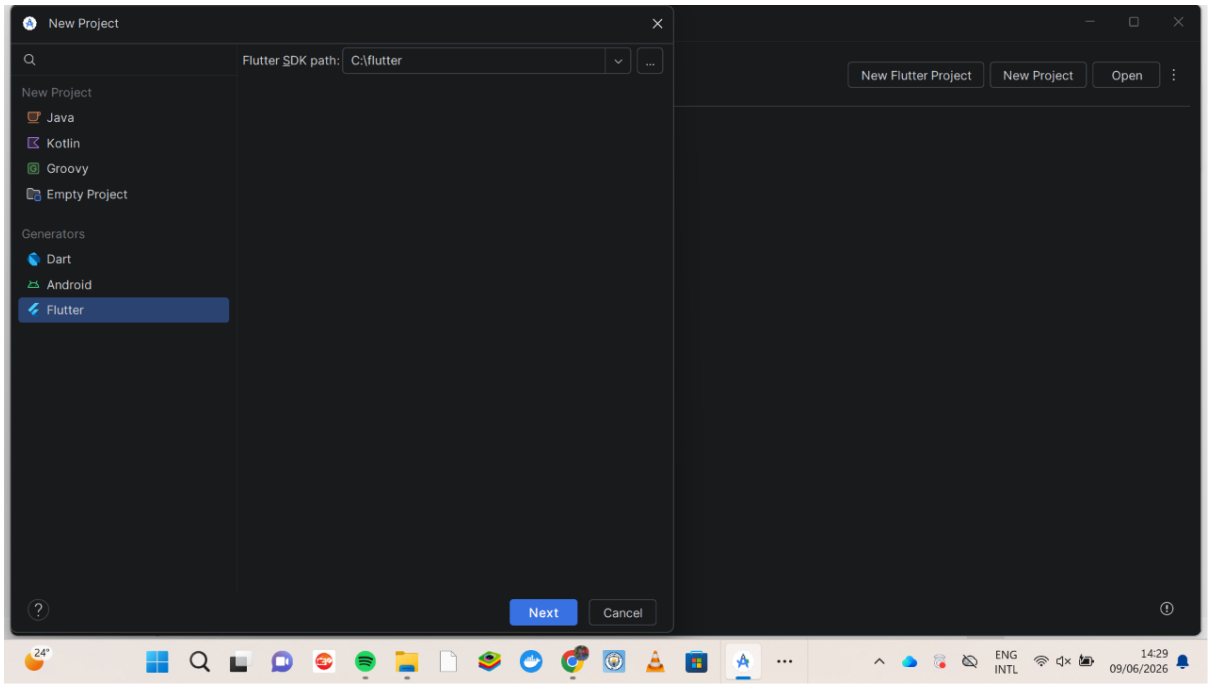
Licence agreement setups

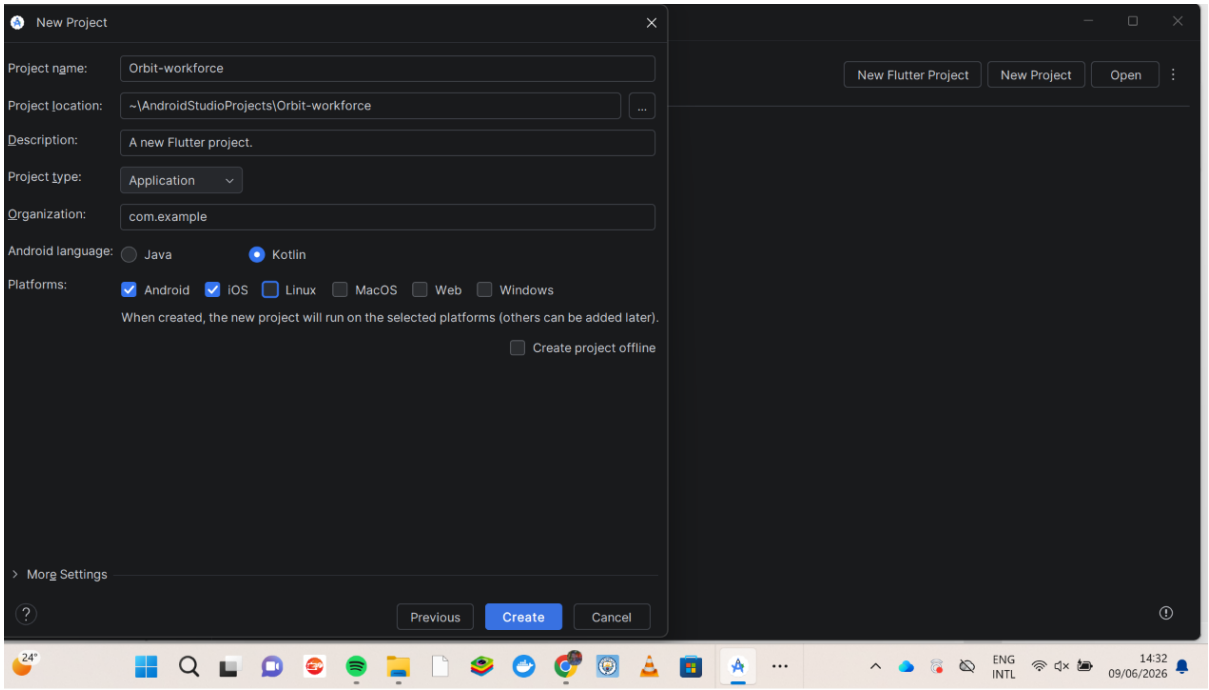


Components dowloads

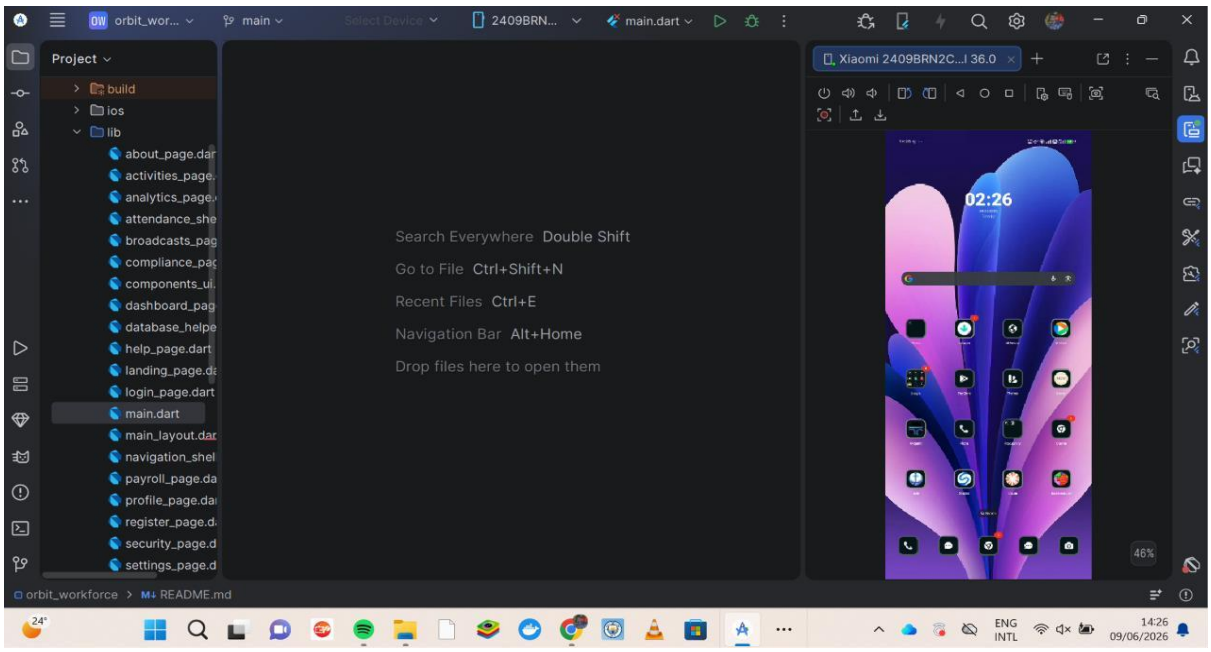


starting new project





Setting up an external phone to be used as an emulator



Tools used for week 1 and week 2

Android studio

Flutter SDK

Flutter plugin

Dart plugin

My personal reflection was how i was able to understand how applications development success really depend on how you start the essential steps at the beginning.

To my own experience setting up the environment wrongly result in much disappointment that may result in stagnation and halt progress definetly.

Week 3: User Interface Design

Task: Development of the Orbit Workforce Landing and Attendance Interface

Design Approach I prioritize a mobile-first approach because I design for workers on the move. I use a clean, professional aesthetic to build trust and ensure the application remains readable in outdoor environments with high glare.

Key Design Decisions

- I implement a consistent visual hierarchy by using large, bold typography for primary actions like "Check-in." This makes it easier for users to interact with the app quickly.
- I design the site selection area using Card widgets. I do this to create clear boundaries between different job locations, preventing user errors when selecting a site.
- I maintain brand consistency by choosing a primary blue color palette, which I find conveys a sense of professionalism and reliability.

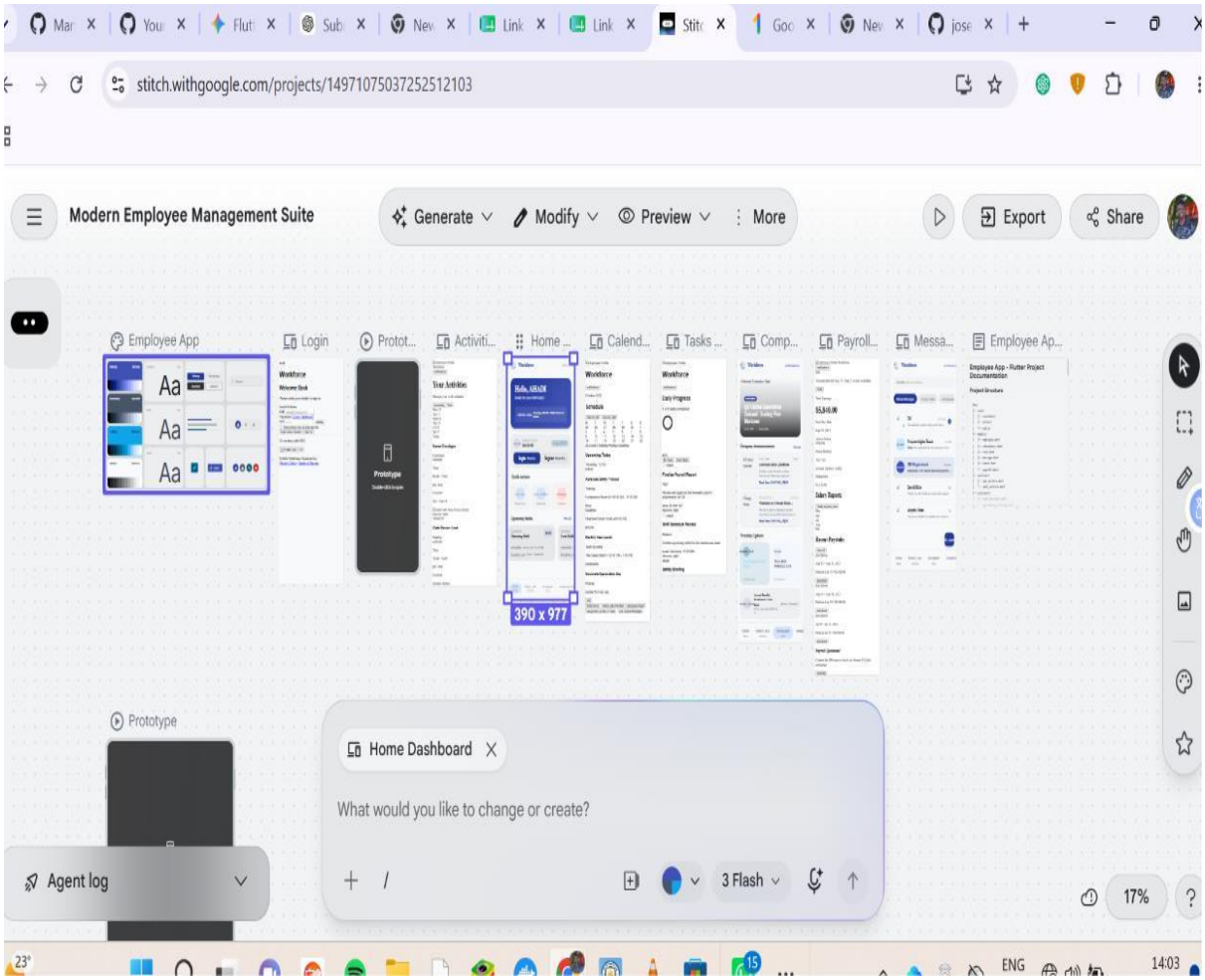
Technical Implementation

- I use the Scaffold widget to provide a stable, standard structure across every page in the app. This ensures that the navigation remains in the same spot, which helps users learn the interface faster.
- I bind the UI to the database by using ListView.builder. I do this to ensure that attendance logs update dynamically whenever a user performs an action, providing them with instant feedback.
- I define all my colors and font sizes in a central theme file. I do this so that I can update the entire app's look and feel from a single location, which saves me time during the development process.

Refinement Process

- I conduct quick usability checks by navigating through the app myself. If I notice a button is hard to press, I increase its hit area to 48x48dp.
- I simplify the registration flow. I notice that too many input fields cause friction, so I remove non-essential requirements to ensure the user reaches the dashboard as quickly as possible.

I USED STITCH TO IMPLEMENT ALL THE USER INTERFACE DESIGNS



After using the Stitch frame work it comes with inbuilt code that can be used in different languages, for my case i had to convert that language to dart so that it can work with flatter effectlively.

- **Week 4: Event Handling and User Interaction**

  I focus on creating an intuitive experience where every user action triggers an immediate and predictable response from the application.

  Steps I Follow for Implementation

1. **I identify the trigger:** First, I determine which user action initiates the event (e.g., a button press, a form submission, or a list item tap).
2. **I define the callback function:** I create a function that houses the logic I want to execute. For example, when a user clicks "Check-in," **I write** an asynchronous function that calls my DatabaseHelper to save the attendance log.
3. **I implement the UI listener:** I wrap my interactive widgets (like ElevatedButton or InkWell) in the appropriate event listener parameters, such as onPressed or onTap.
4. **I manage the state:** Because the app needs to update after an interaction, **I use** setState() or a state management solution. **I trigger** a UI rebuild so the user sees the change immediately (e.g., showing a "Success" message).
5. **I handle user feedback:** I ensure the app tells the user what happened. **I display** a SnackBar or a dialog box to confirm the action, which keeps the user informed and prevents uncertainty.
6. **I include error handling: I wrap** my interaction logic in try-catch blocks. If the database fails or the internet is slow, **I provide** a clear error message to the user instead of letting the app crash.

Week 5

Data Management

The Orbit Workforce Management Platform manages organizational workforce data through structured data models. The system is designed to store and process information related to employees, attendance records, tasks, payroll information, compliance documents, announcements, and user profiles.

Data management ensures that information is organized, accessible, secure, and easy to maintain.

Employee Data

The employee module stores information such as:

- Employee ID

- Full Name

- Email Address

- Phone Number
- Department
- Job Position
- Employment Status

This information is used throughout the system to identify and manage workforce members.

Attendance Data

The attendance module records:

- Clock In Time
- Clock Out Time
- Date
- Employee ID
- Attendance Status

These records help monitor employee presence and support productivity tracking.

Task Data

The task management module stores:

- Task ID
- Task Name
- Task Description
- Assigned Employee
- Due Date
- Task Status

This information supports workforce coordination and task completion monitoring.

Payroll Data

The payroll module manages:

- Employee ID
- Salary Information
- Payroll Period
- Payment Status
- Payslip Details

The payroll records assist in salary administration and payment tracking.

Compliance Data

The compliance module stores:

- Certification Information
- Work Permits
- Expiry Dates
- Compliance Status

This information helps organizations maintain regulatory compliance.

Analytics Data

The analytics module processes data collected from:

- Attendance Records
- Employee Activities
- Task Completion Rates
- Workforce Performance Metrics

The processed information is displayed through reports and dashboards.

Data Validation

The system performs validation to ensure data accuracy.

Examples include:

- Required field validation
- Email format validation
- Password validation

- Date validation

- Numeric field validation

This minimizes data entry errors and improves system reliability.

Week 6

Networking and API Integration

Overview

The Orbit Workforce Management Platform is designed to support communication between the Flutter application and backend services through Application Programming Interfaces (APIs).

APIs enable the application to exchange data with remote servers and cloud databases.

Purpose of APIs

The APIs are responsible for:

- User Authentication

- Employee Data Retrieval

- Attendance Submission

- Task Management

- Payroll Processing

- Compliance Updates

- Analytics Data Collection

Communication Architecture

The application follows a client-server architecture.

Components include:

1. Orbit Mobile app (Client)

2. REST API Service (Backend)

3. Database Server

Workflow:

User Action → obit App → API Request → Server → Database → API Response → Orbit App

API Request Methods

The system supports common HTTP methods:

GET

Used to retrieve information.

Examples:

- Fetch employee list

- Retrieve attendance

POST

Used to create new records.

Examples:

- Register user

- Submit attendance

- Create task

PUT

Used to update existing records.

Examples:

- Update employee information

- Modify task details

DELETE

Used to remove records.

Examples:

- Delete task

- Remove employee record

Security Considerations

To protect organizational data, the system is designed to support:

- Secure Authentication
- Password Encryption
- Access Control
- API Token Authentication
- Secure HTTPS Communication

These mechanisms help prevent unauthorized access to sensitive information.

Future Enhancements

Future versions of the system may include:

- Cloud Database Synchronization incase of reliable financial status
- Real-Time Notifications to ensure communication efficiency
- WebSocket Communication
- Multi-Factor Authentication to enhance clients continuous login and logout
- Advanced API Security Mechanisms

Week 7: Application Architecture

Same as week 1

Week 8: Project Planning

Week 9: Device Features and Sensors

Same as week 1

Week 10: Notifications and Background Services

Same as week 1

Week 11: Mobile Security

Same as week 1

Week 12: Testing and Debugging

Same as week 1

Week 13: Application Deployment

Same as week 1

Week 14: Final Project Presentation

Same as week 1

Semester Project Tracking

Project Title

Project Objectives

Project Requirements

Development Milestones

Testing Results

Lessons Learned

Future Improvements

End of Semester Reflection

Summary of Skills Acquired

Most Interesting Topic

Major Challenges Faced

How Challenges Were Overcome

Professional Growth Achieved

Overall Course Experience

Final Approval

Student Name: _____

Student Signature: _____

Date: _____

Lecturer Name: _____

Lecturer Remarks: _____

Date: _____